GSAS @NIDA : Deep Learning

Deep Learning for Sequence Data 1

Assoc.Prof.Thitirat Siriborvornratanakul, Ph.D.

Email: thitirat@as.nida.ac.th
Website: http://as.nida.ac.th/~thitirat/

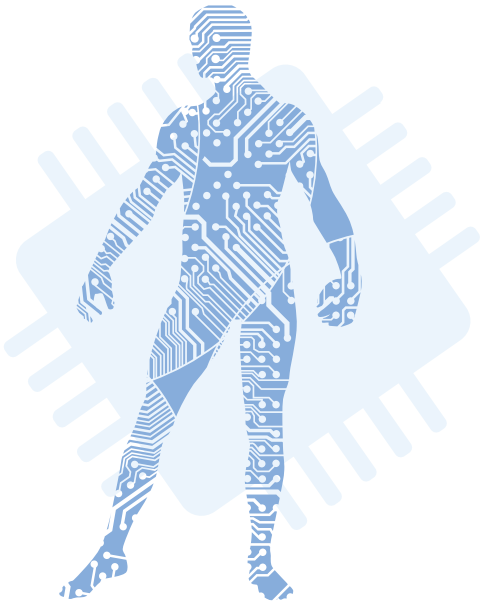
1

OUTLINE

01 Introduction
Sequential inputs and/or outputs in Deep Learning

02 Recurrent Neural Network
The fundamental of vanilla RNN and how to do backpropagation through time

03 Hands-on
Implement various types of RNNs in keras, including a single-layer RNN, stacked RNNs, and Encoder-Decoder RNNs

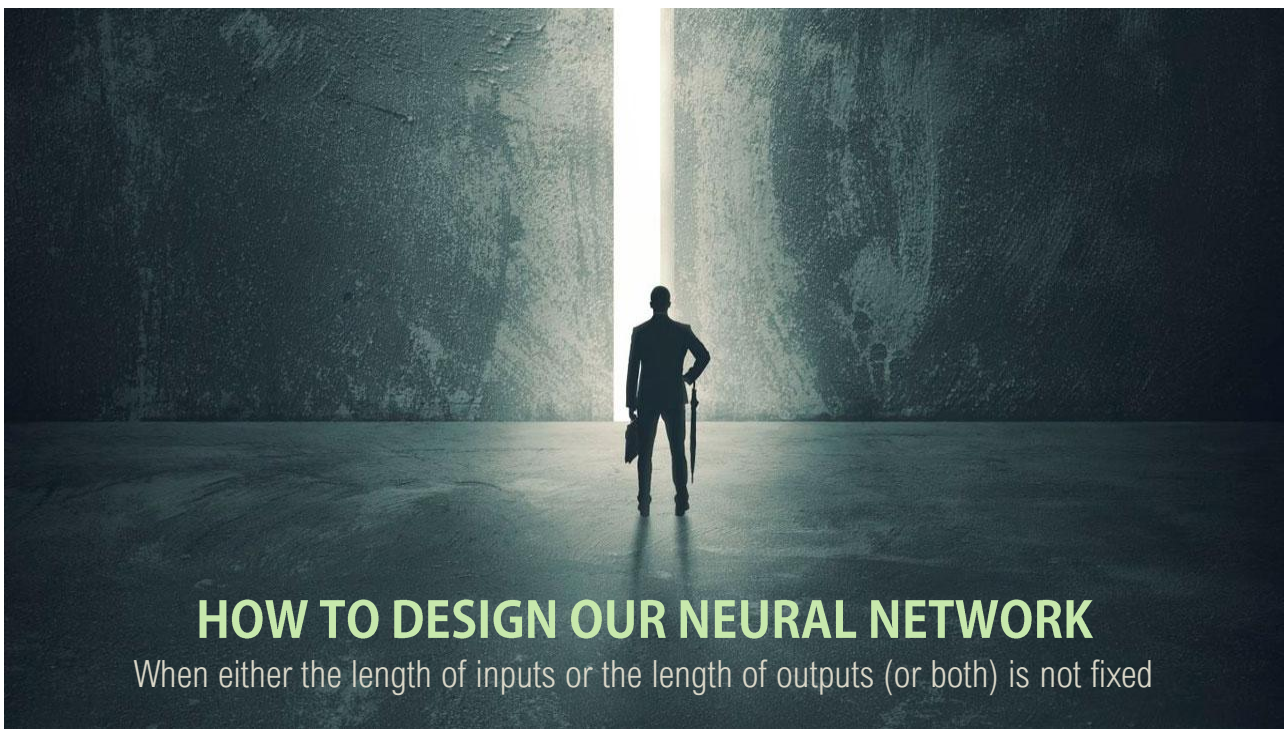


2

Intro

Sequential inputs and/or outputs
in Deep Learning

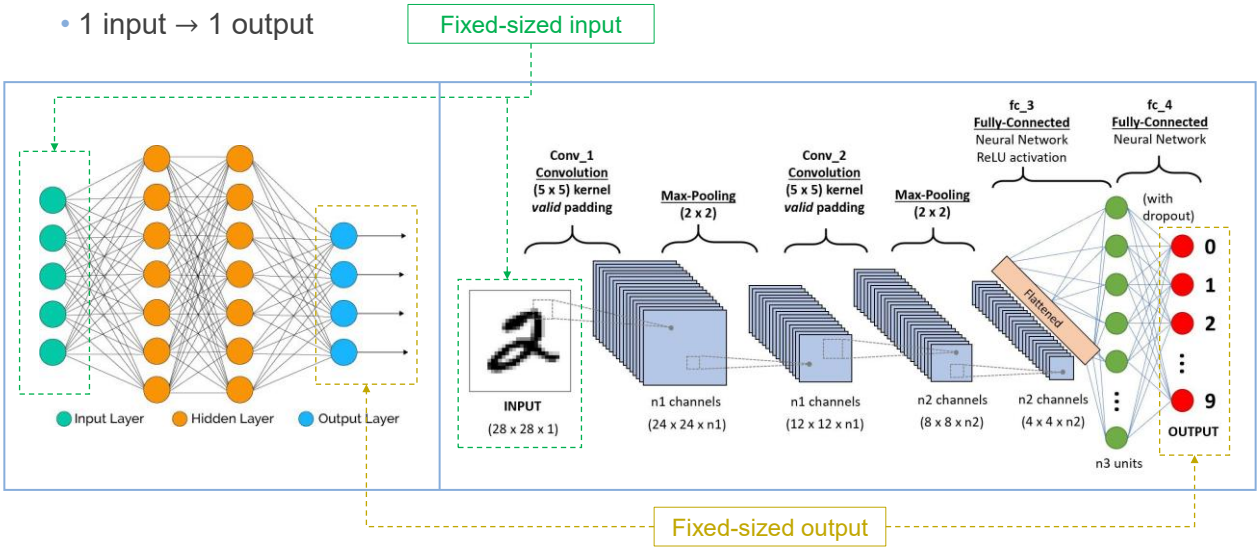
3



4

Non-Sequential Inputs/Outputs

- 1 input → 1 output



5

Limitations of Feed-Forward Network

- Humans don't start their thinking from scratch every second. When reading an essay, we understand each word based on our understanding of previous words. We don't throw everything away and start thinking from scratch again. Our thoughts have persistence.

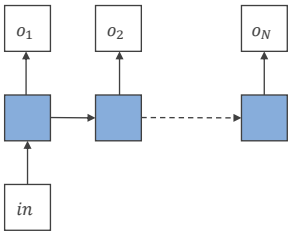
"The concert was boring for the first 15 minutes while the band warmed up but then was terribly exciting."

- Traditional feedforward NNs are not designed to do this as it has no memory and assumes that all inputs (and outputs) are independent of each other.

6

Sequential Inputs/Outputs

- 1 input → **N** outputs



Fixed-sized input

Dynamic-sized output

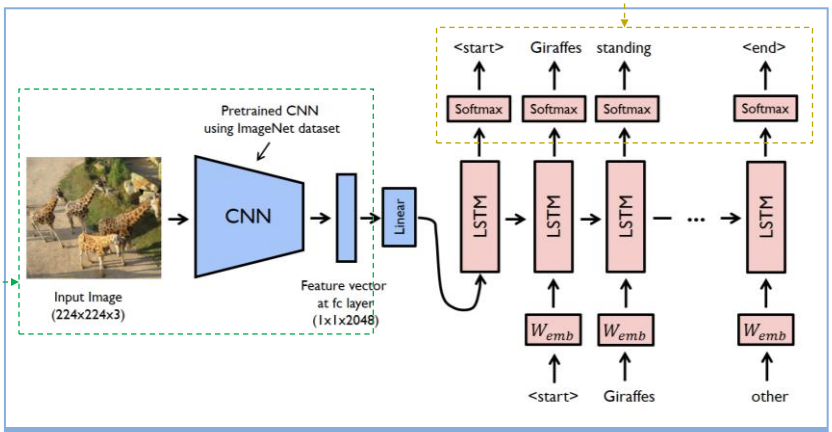
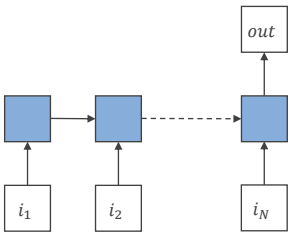


Image captioning

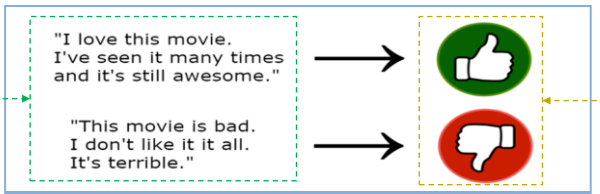
7

Sequential Inputs/Outputs

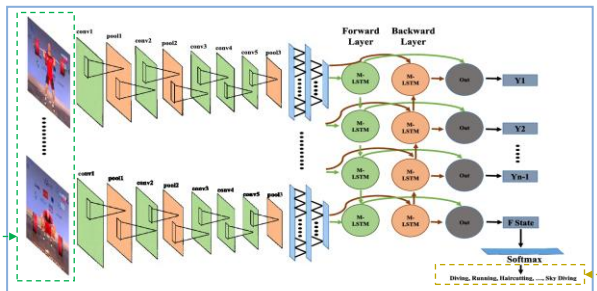
- **N** inputs → 1 output



Dynamic-sized input



Sentiment classification



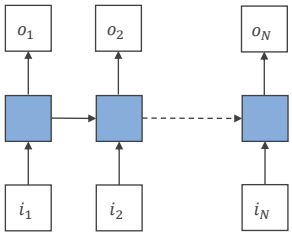
Action classification (in video)

Fixed-sized output

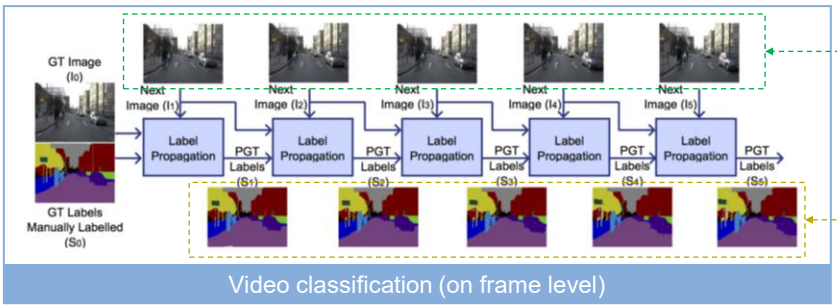
8

Sequential Inputs/Outputs

- **N** inputs → **N** outputs



Synced dynamic-sized input and output



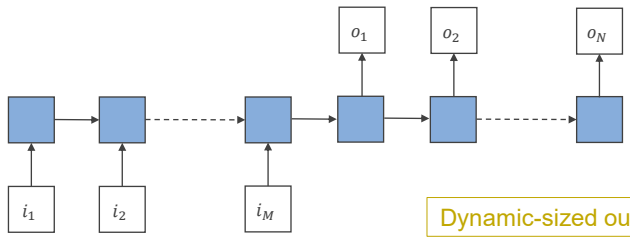
Video classification (on frame level)

9

Sequential Inputs/Outputs

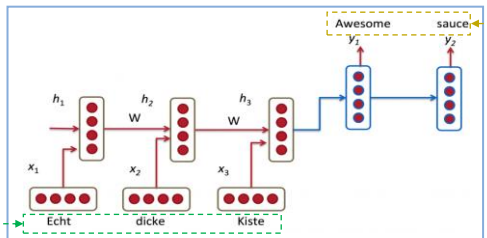
- **M** inputs → **N** outputs

(Encoder-Decoder network)

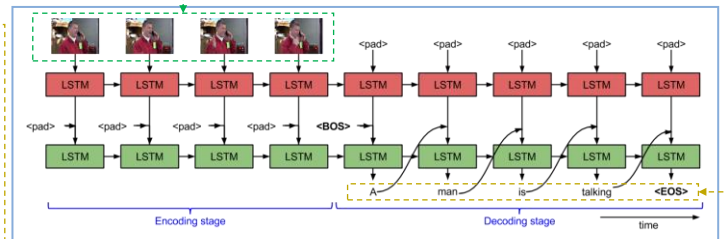


Dynamic-sized input

Dynamic-sized output



Machine Translation

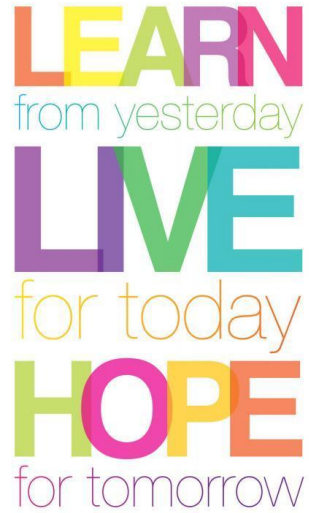


Video Captioning

10

Other Applications

- **Time series data analysis:** stock price prediction
- **Autonomous driving system:** anticipate car trajectories and help avoid accidents
- **Natural Language Processing (NLP):** automatic translation, speech-to-text, sentiment analysis
- **Future prediction (up to a point):** predict the most likely next notes in a melody, generate sentences



11

Dear friends,

I'd like to share a part of the origin story of large language models that isn't widely known. A lot of early work in natural language processing (NLP) was funded by U.S. military intelligence agencies that needed machine translation and speech recognition capabilities. Then, as now, such agencies analyzed large volumes of text and recorded speech in various languages. They poured money into research in machine translation and speech recognition over decades, which motivated researchers to give these applications disproportionate attention relative to other uses of NLP.

This explains why many important technical breakthroughs in NLP stem from studying translation — more than you might imagine based on the modest role that translation plays in current applications. For instance, the celebrated transformer paper, "Attention is All You Need" by the Google Brain team, introduced a technique for mapping a sentence in one language to a translation in another. This laid the foundation for large language models (LLMs) like ChatGPT, which map a prompt to a generated response.

Or consider the BLEU score, which is occasionally still used to evaluate LLMs by comparing their outputs to ground-truth examples. It was developed in 2002 to measure how well a machine-generated translation compares to a ground truth, human-created translation.

DeepLearning.AI

THE BATCH

August 30, 2023 What Matters in AI Right Now

30AUG2023: https://info.deeplearning.ai/text-to-3d-animation-china-restricts-face-recognition-self-driving-cars-get-crash-recorders-1

12

Recommended Reading

Good for RNN's intro

- Colah's blog, Understanding LSTM Networks [27AUG2015] https://colah.github.io/posts/2015-08-Understanding-LSTMs/

Good for RNN's equations and matrices

- Ben Khuong, The Basics of Recurrent Neural Networks (RNNs) [24JUN2019] https://medium.com/towards-artificial-intelligence/whirlwind-tour-of-rnns-a11effb7808f

Good for RNN's applications

- Andrej Karpathy blog, The Unreasonable Effectiveness of Recurrent Neural Networks [21MAY2015] http://karpathy.github.io/2015/05/21/rnn-effectiveness/

13

RNN

Recurrent Neural Network

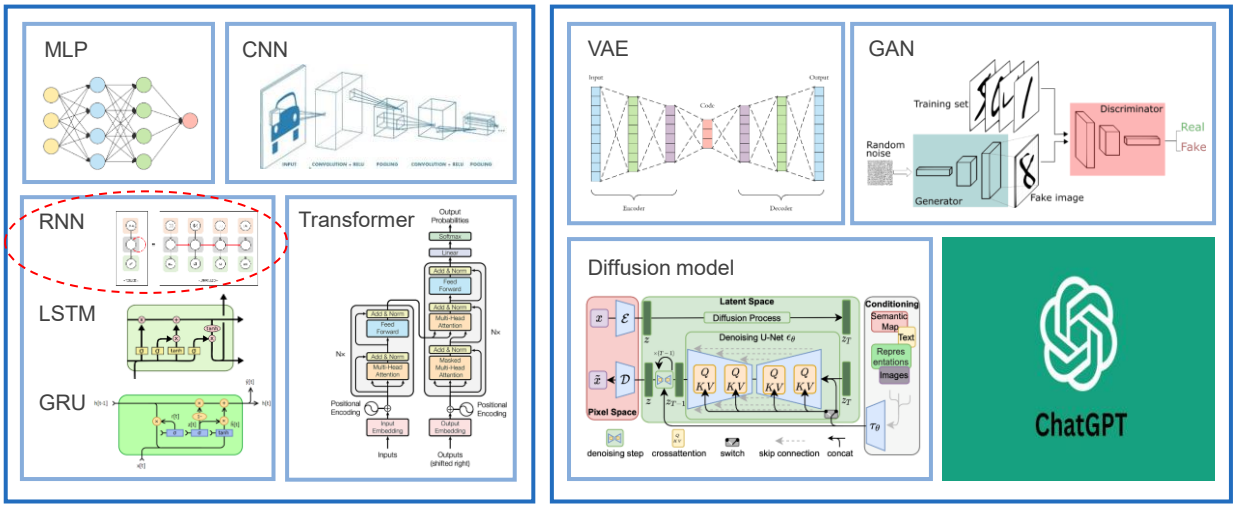
Original RNN paper: D.E. Rumelhart, G.E. Hinton and R.J. Williams, "Learning representations by back-propagating errors," *Nature*, *323*, pp. 533-536, 1986. https://www.nature.com/articles/323533a0

14

DL models in this course

~~DISCRIMINATIVE~~ FUNDAMENTAL

GENERATIVE



15

THE OLD HISTORY

- **1943:** The first neural network was proposed by McCulloch and Pitts.
- **1955:** John McCarthy first coined the term Artificial Intelligence.
- **1959:** Samuel coined and popularized the term Machine Learning.

- **1986:** Birth of backpropagation
- **1986:** Birth of RNN and AE
- **1989:** Birth of CNN
- **1997:** Birth of LSTM

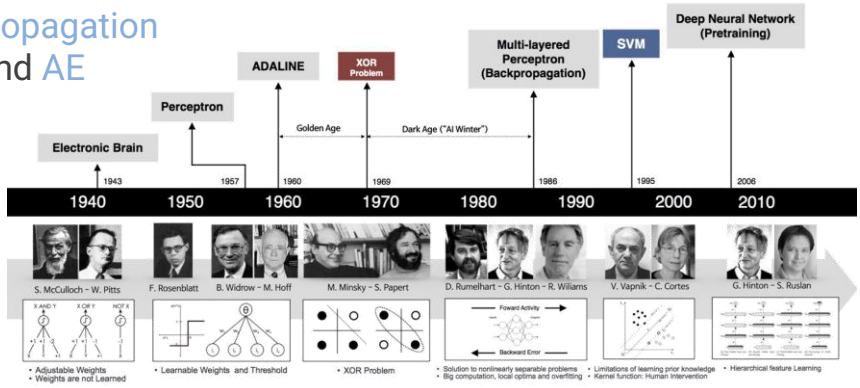
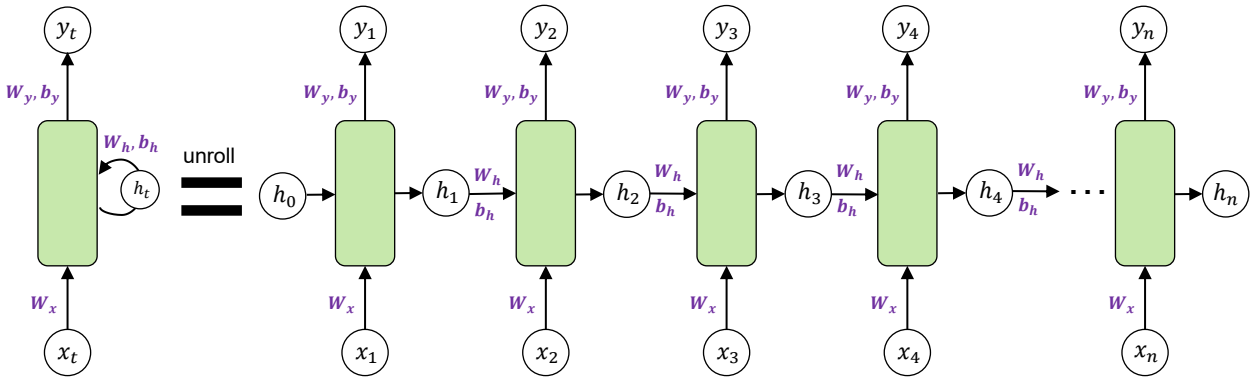


Image credit:
https://beamandrew.github.io/deeplearning/2017/02/23/deep_learning_101_part1.html

16

The Vanilla RNN (Nature 1986)

- **RNNs** (Fully-connected RNNs) are networks with loops in them, allowing information to persist.

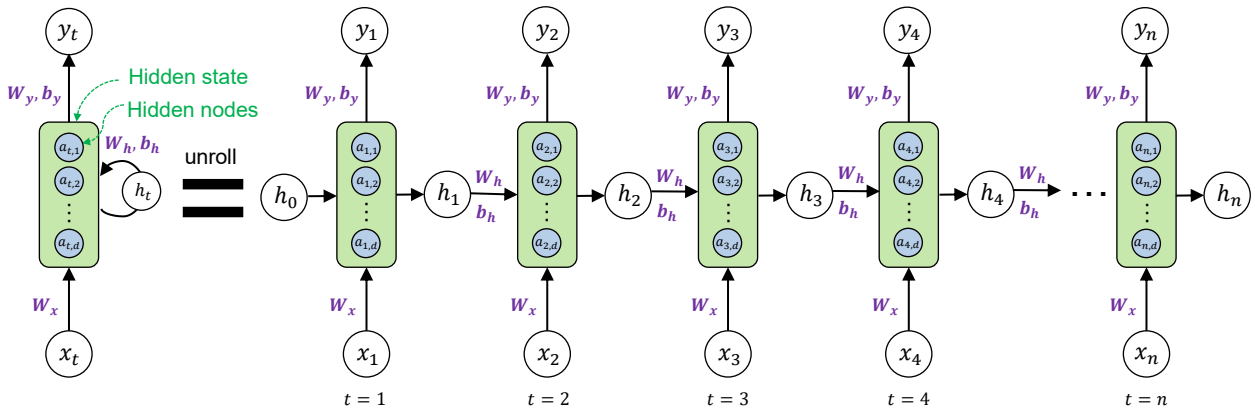


- **RNN** can be thought of as multiple copies of the same network, each passing a message to a successor.

17

The Vanilla RNN (Nature 1986)

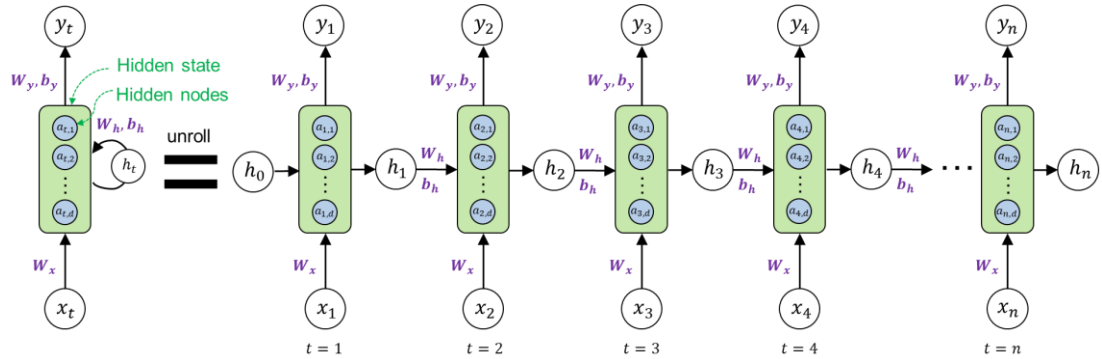
- Similar to activations in MLPs, think of each green block (hidden state) as an activation function that acts on each blue node (hidden node).
- The number of hidden nodes (a.k.a. hidden units, neurons) is decided by the hyper-parameter $\boldsymbol{d}$.



18

RNN's architecture breakdown

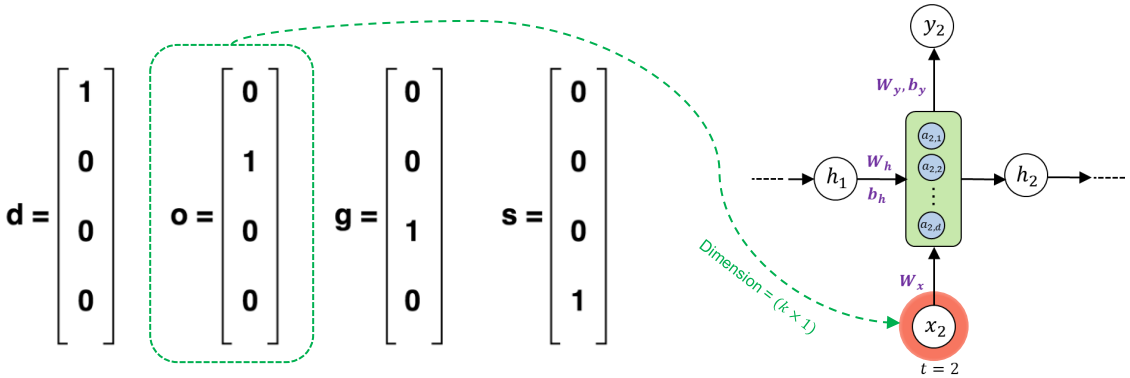
- **Matrices W_x, W_y, W_h (, b_y, b_h)** — are the weights (and biases) of the RNN architecture which are shared throughout the entire network.
  - Ex The model weights of W_x at $t = 1$ are the exact same as the weights of W_x at $t = 2$ and every other time step.



19

RNN's architecture breakdown

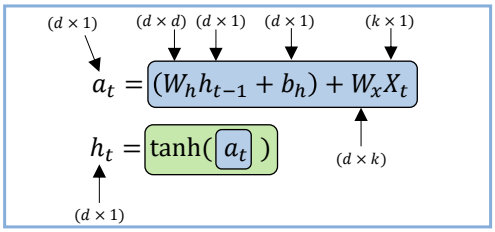
- **Vector x_t** — is the input to each hidden state where $t = 1, 2, \ldots, n$ for each element in the input sequence.
  - Recall that text must be encoded into numerical values. For example, every letter in the word "dogs" would be a one-hot encoded vector with dimension (4x1). Similarly, x can also be word embedding or other numerical representations.

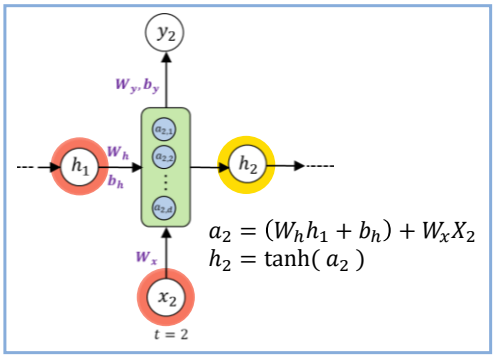


20

RNN's architecture breakdown

- **Vector h** — is the output of the hidden state after the activation function has been applied to the hidden nodes.
  - At time t, the architecture takes into account what happened at $t-1$ by including the h from the previous hidden state as well as the input x at time t.
  - This allows the network to account for information from previous inputs that are sequentially behind the current input.
  - Note that the 0th h vector will always start as a vector of 0's because the algorithm has no information preceding the first element in the sequence.

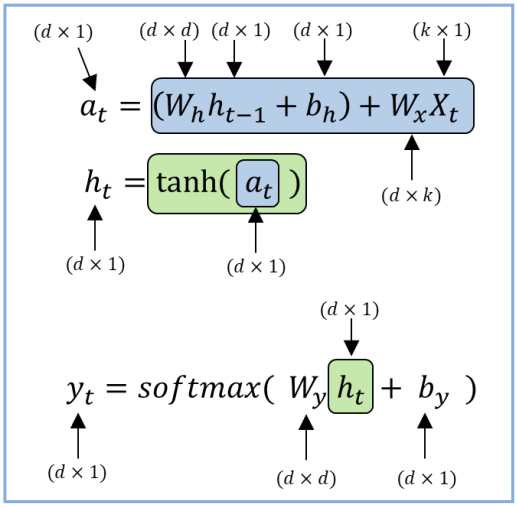


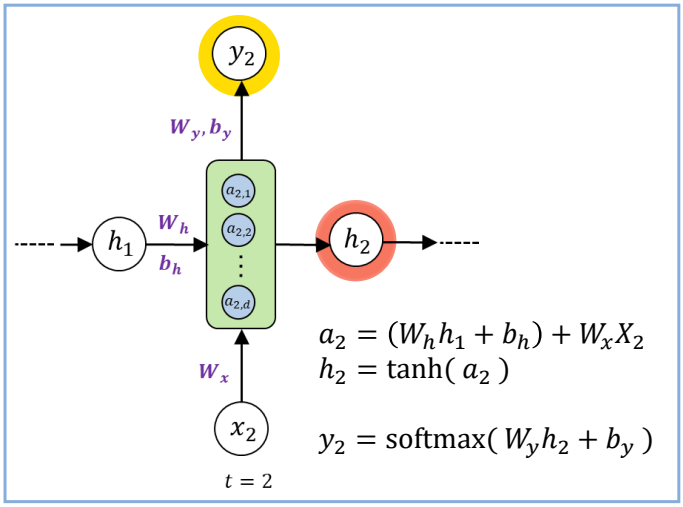


21

RNN's architecture breakdown

- **Vector y_t** — is the predicted value at time t.



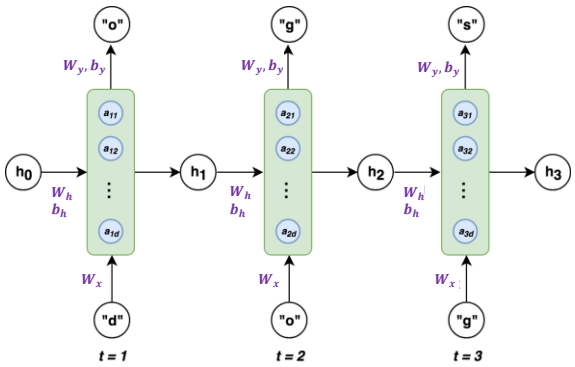


22

RNN's equation: Example

EX. Train RNN to predict the letter **"s"** given the letters **"d"-"o"-"g"**.

- $k = 4$: the input x is a 4-dimensional one-hot vector for the letters in "dogs."
- $d = 3$: here we use 3 hidden nodes for simplicity.



Adapt from: https://medium.com/towards-artificial-intelligence/whirlwind-tour-of-rnns-a11effb7808f

23

RNN's equation: Example

- Forward propagation: $t = 1$ (initialized random weights for all trainable matrices)

$$a_1 = \begin{pmatrix} W_{h,11} & W_{h,12} & W_{h,13} \\ W_{h,21} & W_{h,22} & W_{h,23} \\ W_{h,31} & W_{h,32} & W_{h,33} \end{pmatrix} \underbrace{\begin{pmatrix} h_{0,1} \\ h_{0,2} \\ h_{0,3} \end{pmatrix}}_{h_0 \text{ (zero)}} + \begin{pmatrix} b_{h,1} \\ b_{h,2} \\ b_{h,3} \end{pmatrix} + \begin{pmatrix} W_{x,11} & W_{x,12} & W_{x,13} & W_{x,14} \\ W_{x,21} & W_{x,22} & W_{x,23} & W_{x,14} \\ W_{x,31} & W_{x,32} & W_{x,33} & W_{x,14} \end{pmatrix} \underbrace{\begin{pmatrix} x_{1,1} \\ x_{1,2} \\ x_{1,3} \\ x_{1,4} \end{pmatrix}}_{\text{Input } x_1} = \begin{pmatrix} a_{1,1} \\ a_{1,2} \\ a_{1,3} \end{pmatrix}$$

$$h_1 = \tanh\left(\begin{pmatrix} a_{1,1} \\ a_{1,2} \\ a_{1,3} \end{pmatrix}\right) = \begin{pmatrix} h_{1,1} \\ h_{1,2} \\ h_{1,3} \end{pmatrix}$$

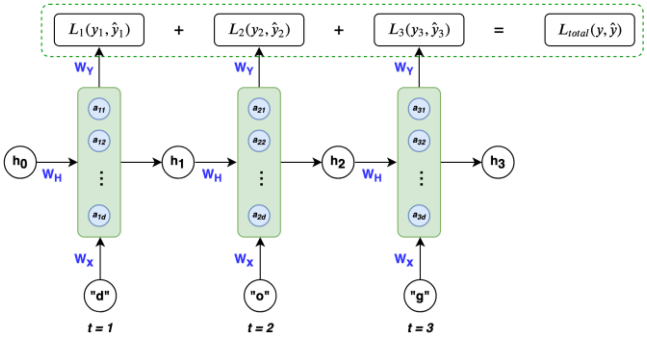
$$y_1 = softmax\left(\begin{pmatrix} W_{y,11} & W_{y,12} & W_{y,13} \\ W_{y,21} & W_{y,22} & W_{y,23} \\ W_{y,31} & W_{y,32} & W_{y,33} \\ W_{y,41} & W_{y,42} & W_{y,43} \end{pmatrix} \begin{pmatrix} h_{1,1} \\ h_{1,2} \\ h_{1,3} \end{pmatrix} + \begin{pmatrix} b_{y,1} \\ b_{y,2} \\ b_{y,3} \\ b_{y,4} \end{pmatrix}\right) = \underbrace{\begin{pmatrix} y_{1,1} \\ y_{1,2} \\ y_{1,3} \\ y_{1,4} \end{pmatrix}}_{\text{Output } y_1}$$

- Forward propagation: at $t = 2$ and $t = 3$,
  - The workflow would be analogous except that the vector h from $t - 1$ would no longer be a vector of 0's, but a vector of non-zeros based on the inputs before time t.
  - As a reminder, all trainable matrices remain the same for $t = 1, 2,$ and 3.

24

Backpropagation Through Time (BPTT)

- Backpropagation with RNNs is a little more challenging due to the recursive nature of the weights and their effect on the loss which spans over time.
- Note that the output h from the hidden unit is not learned, it is merely the information gained by concatenating the learned weights to previous output h and current input x.



Categorical cross entropy loss function:

$$L_t(y_t, \hat{y}_t) = -y_t log(\hat{y}_t)$$

$$L_{total}(y, \hat{y}) = -\sum_{t=1}^{n} y_t log(\hat{y}_t)$$

Calculate the gradients for our three weight matrices W_x, W_y, W_h, and update them with a learning rate η:

$$W_i := W_i - \eta \frac{\partial L_{total}(y, \hat{y})}{\partial W_i}$$

Credit: https://medium.com/towards-artificial-intelligence/whirlwind-tour-of-rnns-a11effb7808f

25

Backpropagation Through Time (BPTT)

- The weight gradient for W_y,
  - Function dependencies with respect to W_y,

$$L_t = -y_t log(\hat{y}_t) \rightarrow \hat{y}_t = softmax(z_t) \rightarrow z_t = W_Y h_t$$

  - Chain rule with respect to W_y,

$$\frac{\partial L_t}{\partial W_Y} = \frac{\partial L_t}{\partial \hat{y}_t} \frac{\partial \hat{y}_t}{\partial z_t} \frac{\partial z_t}{\partial W_Y}$$

  - Gradient for W_y,

$$\frac{\partial L_{total}}{\partial W_Y} = \frac{\partial L_1}{\partial \hat{y}_1} \frac{\partial \hat{y}_1}{\partial z_1} \frac{\partial z_1}{\partial W_Y} + \frac{\partial L_2}{\partial \hat{y}_2} \frac{\partial \hat{y}_2}{\partial z_2} \frac{\partial z_2}{\partial W_Y} + \frac{\partial L_3}{\partial \hat{y}_3} \frac{\partial \hat{y}_3}{\partial z_3} \frac{\partial z_3}{\partial W_Y} = \sum_{t=1}^{n} \frac{\partial L_t}{\partial W_Y}$$

Credit: https://medium.com/towards-artificial-intelligence/whirlwind-tour-of-rnns-a11effb7808f

26

Backpropagation Through Time (BPTT)

- The weight gradient for W_x :
  - Function dependencies with respect to W_x,

$$L_t = -y_t log(\hat{y}_t) \rightarrow \hat{y}_t = softmax(z_t) \rightarrow z_t = W_Y h_t \rightarrow h_t = tanh(W_H h_{t-1} + W_X X_t) \quad (1)$$

  - Chain rule with respect to W_x,

$$\frac{\partial L_t}{\partial W_x} = \frac{\partial L_t}{\partial \hat{y}_t}\frac{\partial \hat{y}_t}{\partial z_t}\frac{\partial z_t}{\partial h_t}\frac{\partial h_t}{\partial W_x}$$ but note, that within h_t, h_{t-1} also contains W_x, thus we need to recurisvely chain rule h_{t-1} until we reach h_0 (2)

$$\frac{\partial L_t}{\partial W_x} = \frac{\partial L_t}{\partial \hat{y}_t}\frac{\partial \hat{y}_t}{\partial z_t}\frac{\partial z_t}{\partial h_t}\frac{\partial h_t}{\partial W_x} + \frac{\partial L_t}{\partial \hat{y}_t}\frac{\partial \hat{y}_t}{\partial z_t}\frac{\partial z_t}{\partial h_t}\frac{\partial h_t}{\partial h_{t-1}}\frac{\partial h_{t-1}}{\partial W_x} + \ldots + \frac{\partial L_t}{\partial \hat{y}_t}\frac{\partial \hat{y}_t}{\partial z_t}\frac{\partial z_t}{\partial h_t}\frac{\partial h_t}{\partial h_{t-n}}\frac{\partial h_{t-n}}{\partial W_x} = \sum_{k=0}^{n}\frac{\partial L_t}{\partial \hat{y}_t}\frac{\partial \hat{y}_t}{\partial z_t}\frac{\partial z_t}{\partial h_t}\frac{\partial h_t}{\partial h_k}\frac{\partial h_k}{\partial W_x} \quad (3)$$

This formula refers to **multivariable chain rule**. For more information, please continue reading in "*The Matrix Calculus You Need For Deep Learning*" https://arxiv.org/abs/1802.01528 .

  - Gradient for W_x,

$$\frac{\partial L_{total}}{\partial W_X} = \sum_{t=1}^{n}\sum_{k=0}^{n}\frac{\partial L_t}{\partial \hat{y}_t}\frac{\partial \hat{y}_t}{\partial z_t}\frac{\partial z_t}{\partial h_t}\frac{\partial h_t}{\partial h_k}\frac{\partial h_k}{\partial W_x} \quad (4)$$

Credit: https://medium.com/towards-artificial-intelligence/whirlwind-tour-of-rnns-a11effb7808f

27

Actual RNN's implementation

```
[1] import tensorflow as tf
    print( f"TensorFlow {tf.__version__}\n" )

    rnn_keras = tf.keras.models.Sequential( [tf.keras.layers.SimpleRNN(10, input_shape=(None,1))] )
    rnn_keras.summary()

    for i,w in enumerate(rnn_keras.layers[0].weights):
      print( f"[{i}] {w.name} , {w.shape}" )
```

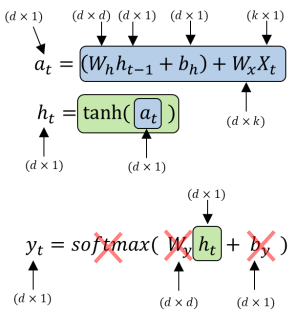
```
TensorFlow 2.8.2

Model: "sequential"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 simple_rnn (SimpleRNN)      (None, 10)                120

=================================================================
Total params: 120
Trainable params: 120
Non-trainable params: 0
_________________________________________________________________
[0] simple_rnn/simple_rnn_cell/kernel:0 , (1, 10)
[1] simple_rnn/simple_rnn_cell/recurrent_kernel:0 , (10, 10)
[2] simple_rnn/simple_rnn_cell/bias:0 , (10,)
```

W_x (input weights)

W_h (recurrent weights)

b_h (biases)

Keras



28

Actual RNN's implementation

```
import torch
import torchvision
print(f"Torch {torch.__version__}, TorchVision {torchvision.__version__}\n")

rnn_pytorch = torch.nn.RNN(input_size=1, hidden_size=10, num_layers=1)
for i, (name,param) in enumerate(rnn_pytorch.named_parameters()):
  print(f"[{i}] {name} {param.shape}")
```

```
Torch 1.12.1+cu113, TorchVision 0.13.1+cu113

[0] weight_ih_l0 torch.Size([10, 1])
[1] weight_hh_l0 torch.Size([10, 10])
[2] bias_ih_l0 torch.Size([10])
[3] bias_hh_l0 torch.Size([10])
```

- W_x (input weights)
- W_h (recurrent weights)
- b_x (biases)
- b_h (biases): *"The second bias vector included for CuDNN compatibility. Only one bias vector is needed in standard definition."*

Ref: https://pytorch.org/docs/stable/_modules/torch/nn/modules/rnn.html#RNN

PyTorch

$$a_t = (W_h h_{t-1} + b_h) + W_x X_t \qquad h_t = \tanh(a_t)$$

$$y_t = softmax(W_y h_t + b_y)$$

29

Regularization in RNN

```
keras.layers.SimpleRNN(
    units,
    activation="tanh",
    use_bias=True,
    kernel_initializer="glorot_uniform",
    recurrent_initializer="orthogonal",
    bias_initializer="zeros",
    kernel_regularizer=None,
    recurrent_regularizer=None,
    bias_regularizer=None,
    activity_regularizer=None,
    kernel_constraint=None,
    recurrent_constraint=None,
    bias_constraint=None,
    dropout=0.0,
    recurrent_dropout=0.0,
    return_sequences=False,
    return_state=False,
    go_backwards=False,
    stateful=False,
    unroll=False,
    seed=None,
    **kwargs
)
```

- Quoted from our 3rd class on Dropout: "p is a new hyperparameter called the dropout rate, and typically set between 10-50% (20-30% in RNN and 40-50% in CNN)."

`kernel_regularizer`	Regularizer function applied to the `kernel` weights matrix. Default: `None`.
`recurrent_regularizer`	Regularizer function applied to the `recurrent_kernel` weights matrix. Default: `None`.
`bias_regularizer`	Regularizer function applied to the bias vector. Default: `None`.
`activity_regularizer`	Regularizer function applied to the output of the layer (its "activation"). Default: `None`.
`dropout`	Float between 0 and 1. Fraction of the units to drop for the linear transformation of the inputs. Default: 0.
`recurrent_dropout`	Float between 0 and 1. Fraction of the units to drop for the linear transformation of the recurrent state. Default: 0.

30

Regularization in RNN

```
keras.layers.SimpleRNN(
    units,
    activation="tanh",
    use_bias=True,
    kernel_initializer="glorot_uniform",
    recurrent_initializer="orthogonal",
    bias_initializer="zeros",
    kernel_regularizer=None,
    recurrent_regularizer=None,
    bias_regularizer=None,
    activity_regularizer=None,
    kernel_constraint=None,
    recurrent_constraint=None,
    bias_constraint=None,
    dropout=0.0,
    recurrent_dropout=0.0,
    return_sequences=False,
    return_state=False,
    go_backwards=False,
    stateful=False,
    unroll=False,
    seed=None,
    **kwargs
)
```

- **BUT** according to the ICLR2015 paper by NY University & Google: https://arxiv.org/abs/1409.2329 (citation 3950+ on 2/2025)
  - Application of dropout to feedforward neural networks gives promising results.
  - Applying dropout in the standard manner (dropout outside the RNN layer) to RNNs fails to give any improvement.
    - When dropout is used for regularizing RNNs, the 'disturbance' it generates at each time step propagates over a long interval, thereby decreasing the network's ability to represent long-range dependencies.
  - The connections between cells in the same layer are recurrent connections, and those between cells in adjacent layers are non-recurrent connections. This work suggested that dropout should be applied only to non-recurrent connections.
- Continue reading about dropout in RNN:
  - https://adriangcoder.medium.com/a-review-of-dropout-as-applied-to-rnns-72e79ecd5b7b
  - https://stackoverflow.com/questions/50458428/about-correctly-using-dropout-in-rnns-keras

31

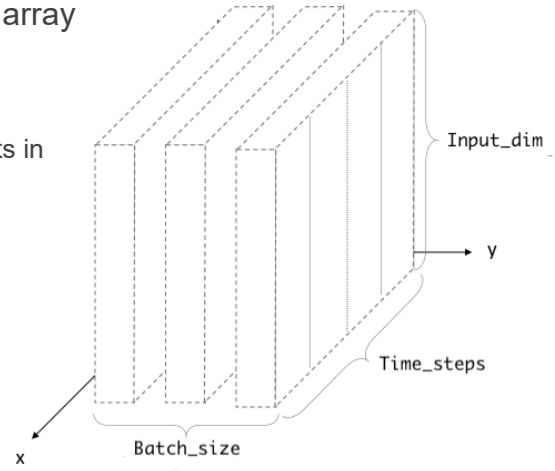
HANDS ON

Simple RNN in Keras

32

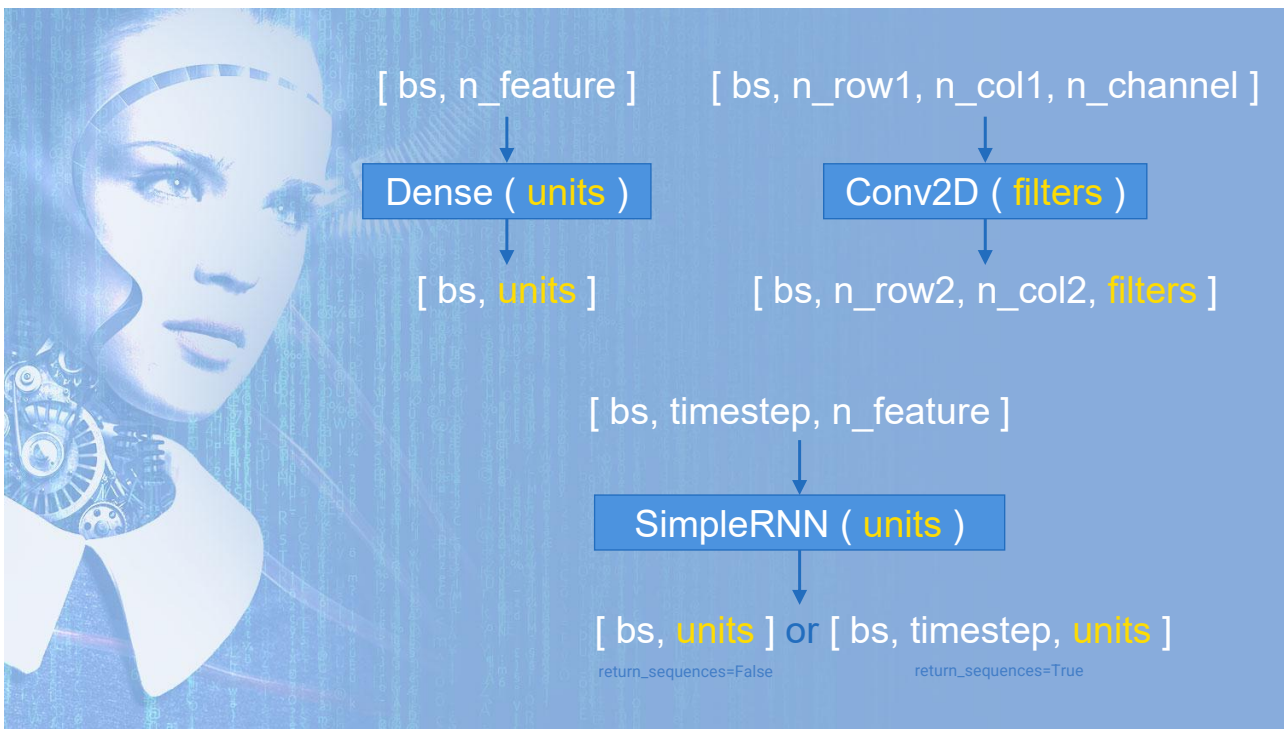
Input/output shapes of RNN (Keras)

- We always have to give a three-dimensional array as an input to `keras.layers.SimpleRNN`.
  - 1st dimension: `batch_size`
  - 2nd dimension: `time_steps`
  - 3rd dimension: `input_dim` (the number of units in one input sequence)
- The input shape looks like `(batch_size, time_steps, input_dim)`.



Credit: https://medium.com/@shivajbd/understanding-input-and-output-shape-in-lstm-keras-c501ee95c65e

33

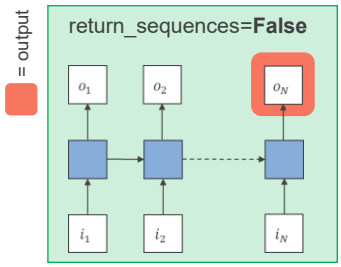


34

The number of neurons. Dimensionality of the output space.

The default activation function applied to hidden states.

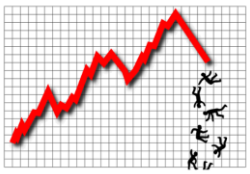
```
keras.layers.SimpleRNN ( units,
                         activation='tanh',
                         return_sequences=False
                       )
```



35

QUICK OVERVIEW: Time series analysis

- How to create a dataset from an array of timeseries?



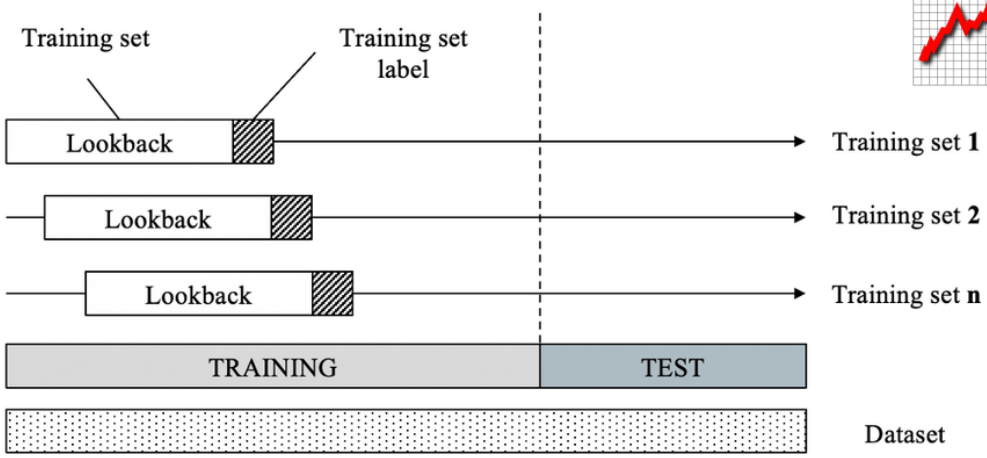
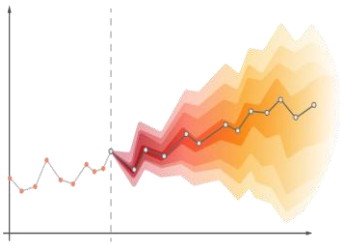


Image credit: https://www.researchgate.net/figure/Illustration-of-the-sliding-window-technique-to-create-input-features-and-labels-from_fig2_348379817

36

QUICK OVERVIEW: Time series analysis

- Be careful of data leakage in dataset splitting!!!
  - Shuffling the whole dataset before splitting it will ensure that our training set's coverage spans across all data possibilities.
  - **BUT** for time series (e.g., stock price), there is dependence from one observation to the other. Hence, it is suggested to use values at the rear of the dataset for testing and everything else for training.



- Can we use RNN to forecast non-stationary (unbound) time series data?
  - Theoretically, yes. However, predicting data outside the training distribution may lead to poor results.
  - Some people suggest doing scaling/detrending to the data before feeding them to the model. This should help reduce network capacity and training.

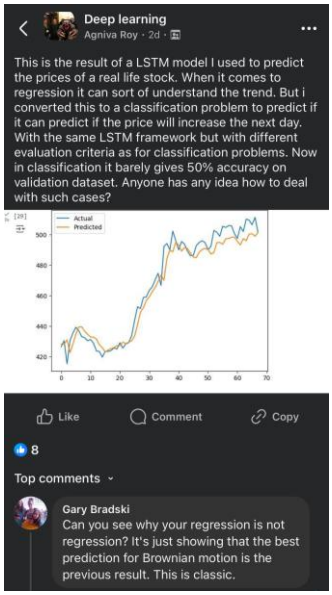
37

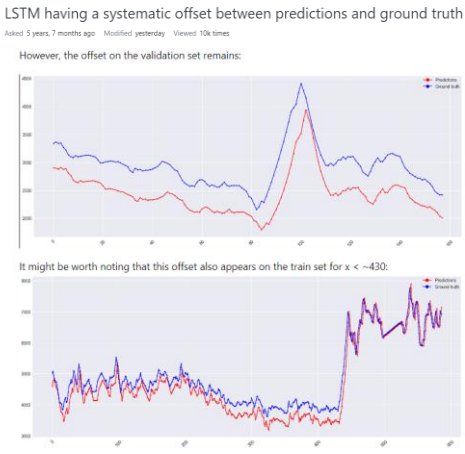
QUICK OVERVIEW: Time series analysis

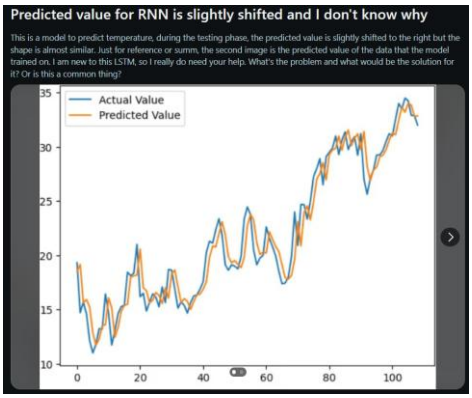
- Time series normalization checklist:
  1. Always use only training statistics. Never use the full dataset to avoid data leakage.
  2. Separate normalizers for inputs (X) and targets (Y) if X and Y have very different scales.
  3. Normalize before sliding windows to ensure that each input sequence is scaled consistently.
  4. Choose the right normalization method:
     - **MinMax scaling** maps values to a fixed range (e.g., [0,1]).
       - Watching for future values outside training min/max.
     - **Z-score (standardization)** scales by training mean and SD.
       - Safer for trending or unbounded sequences.
     - **Per-window normalization** normalizes each sequence individually.
       - Can help with non-stationary data but may hide long-term trends.
  5. Always denormalize predictions (using the same training statistics) to reverse the normalization to the original scale.

38

QUICK OVERVIEW: Time series analysis







9SEP2024 https://www.facebook.com/share/p/2TexzeWnHvSHUaP3/?
16AUG2023 https://www.reddit.com/r/learnmachinelearning/comments/178kxtk/predicted_value_for_rnn_is_slightly_shifted_and_i/?rdt=57926
25JAN2019 https://stackoverflow.com/questions/54368686/lstm-having-a-systematic-offset-between-predictions-and-ground-truth

39

EX1: Guess the next number

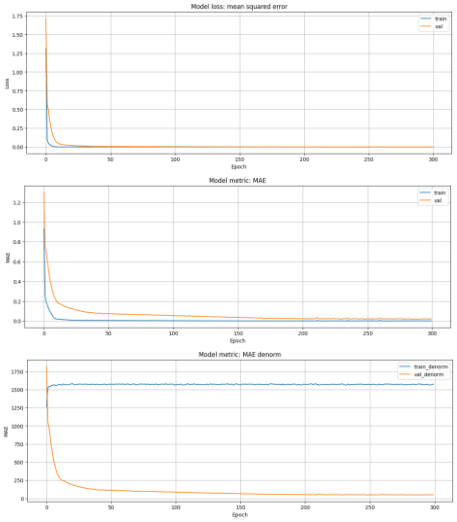
- For this example, we use a simple sequence of $x_n = x_{n-1} + 2$.

- **Summary:**
  - Use a `keras.layers.SimpleRNN` followed by `keras.layers.Dense` for a sequence-based regression problem
  - Use a built-in MAE and a custom denormalized MAE to monitor model performances

40

EX1: Guess the next number

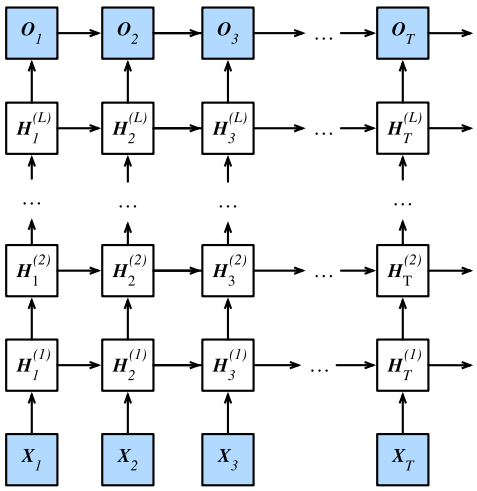
- **Result discussion:**
  - Without denormalization:
    - Training MAE < Validation MAE ✅ (expected — normal behavior)
  - With denormalization:
    - Training MAE ≈ 1500 (huge)
    - Validation MAE ≈ 50 (tiny, flipped trend ❌)
  - Because our training set contains very negative values while validation contains only positives, the same normalized error maps to much larger raw error on train than val.
  - So the flip (train > val after denorm) is not overfitting — it's a scale mismatch problem due to distribution shift.



41

Stacked RNN (Deep RNN)

- A deep recurrent neural network with L hidden layers.
- Each hidden state is continuously passed to both the next time step of the current layer and the current time step of the next layer.

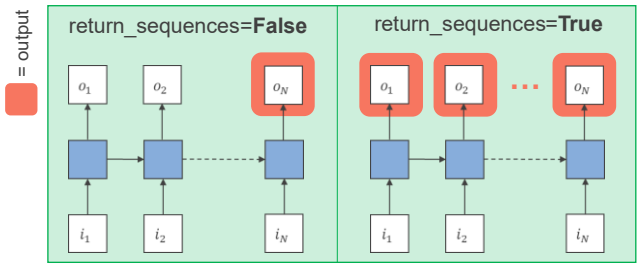


Image(s) from: https://d2l.ai/chapter_recurrent-neural-networks/deep-rnn.html

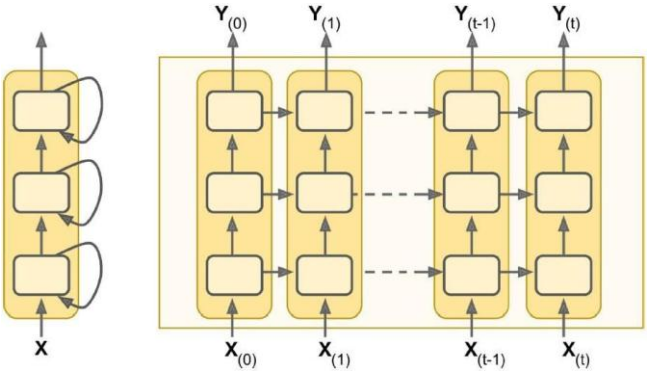
42

By default, SimpleRNN in keras only returns the final output from the last time step. To make it return one output per one time step, just set `return_sequence=True`.

```
keras.layers.SimpleRNN ( units,
                         activation='tanh',
                         return_sequences=False
                       )
```



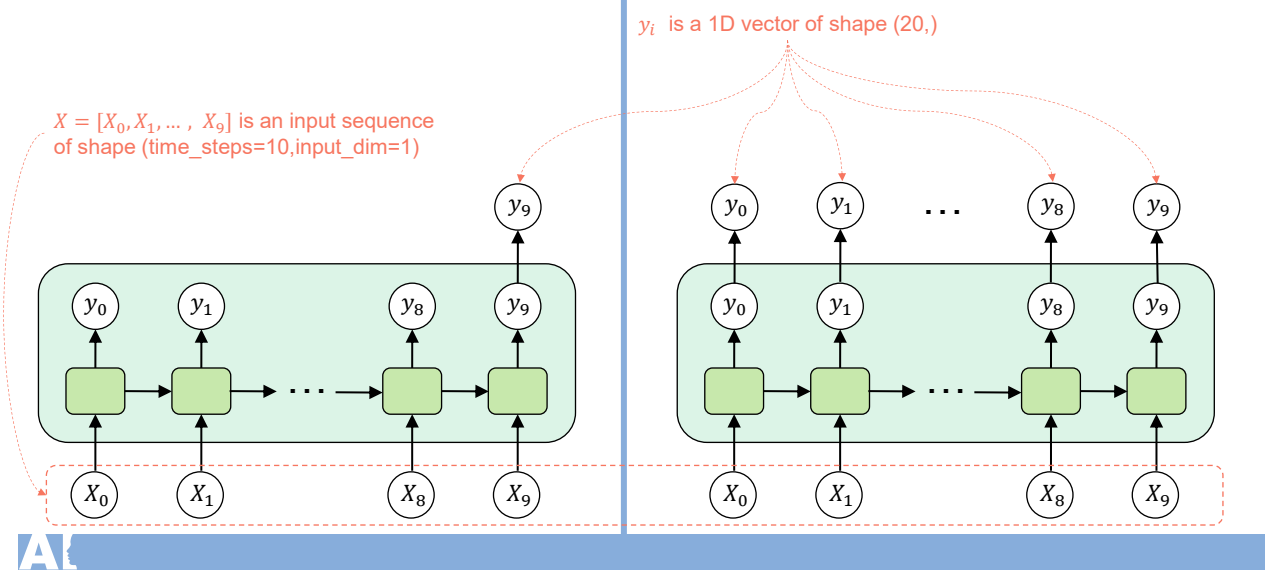
EX2: Guess the next number (revisited)



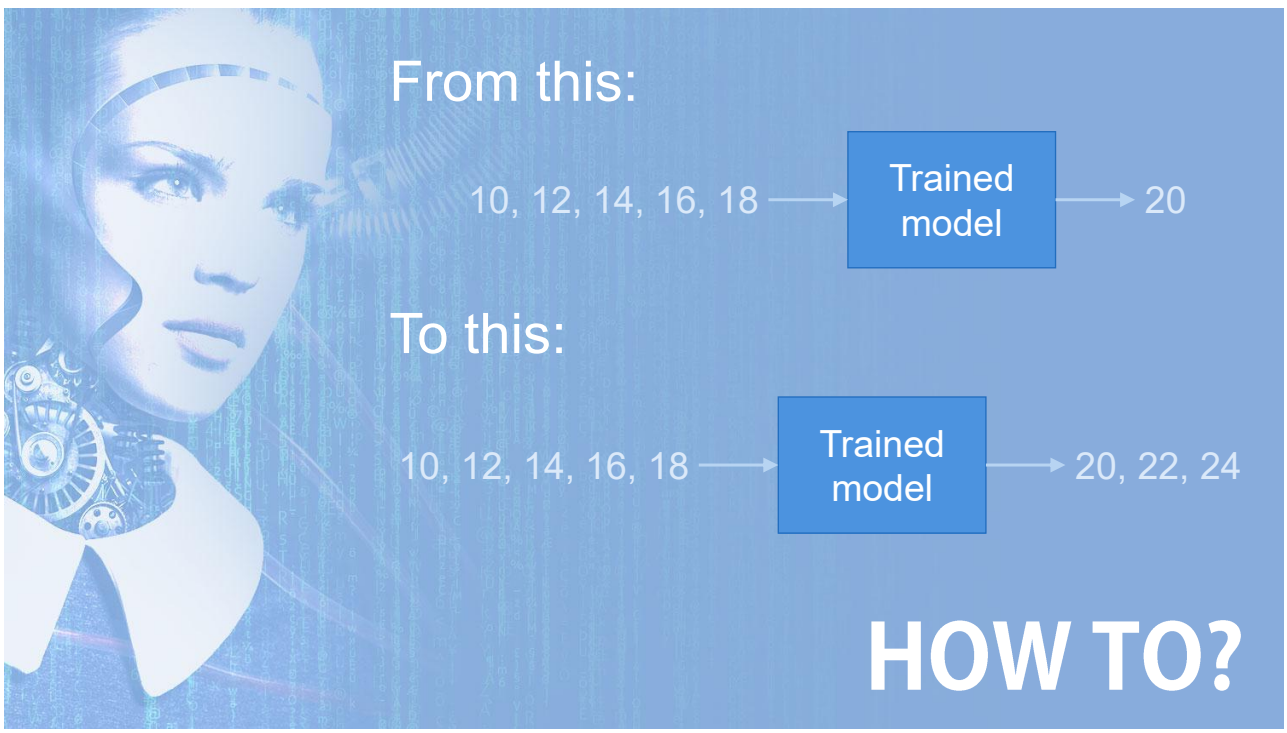
- Summary:
  - Use stacked `keras.layers.SimpleRNN` layers to create a deep RNN model
  - Set `return_sequence=True` in order to output predicted values from all time steps

```
model.add( SimpleRNN( units=20 ) )
```

```
model.add( SimpleRNN( units=20,
                      return_sequences=True ) )
```



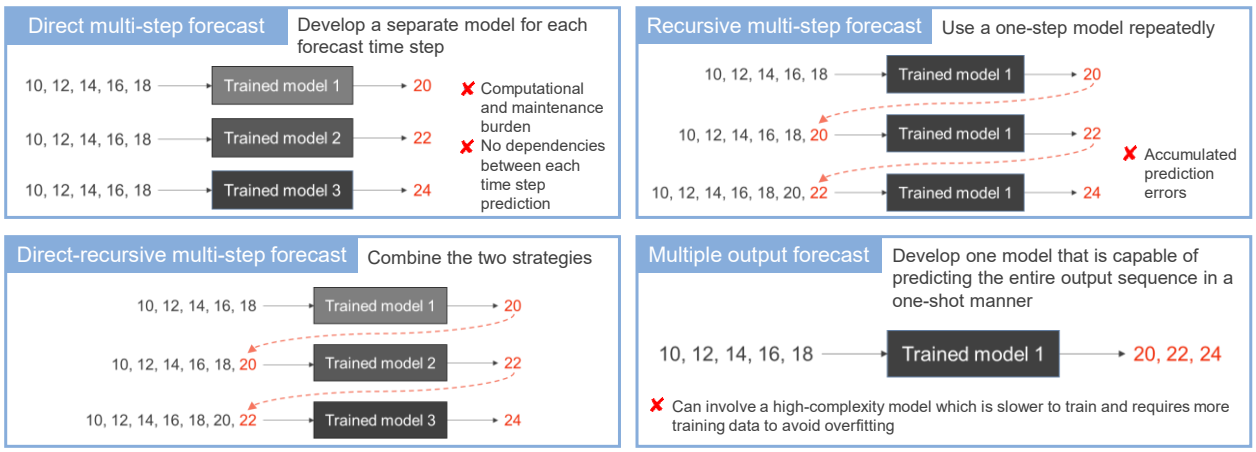
45



46

SEQ2SEQ prediction

- Both input and output are sequences (a.k.a., multi-step time series forecasting).
- Four commonly used strategies: (21AUG2019: https://machinelearningmastery.com/multi-step-time-series-forecasting/)



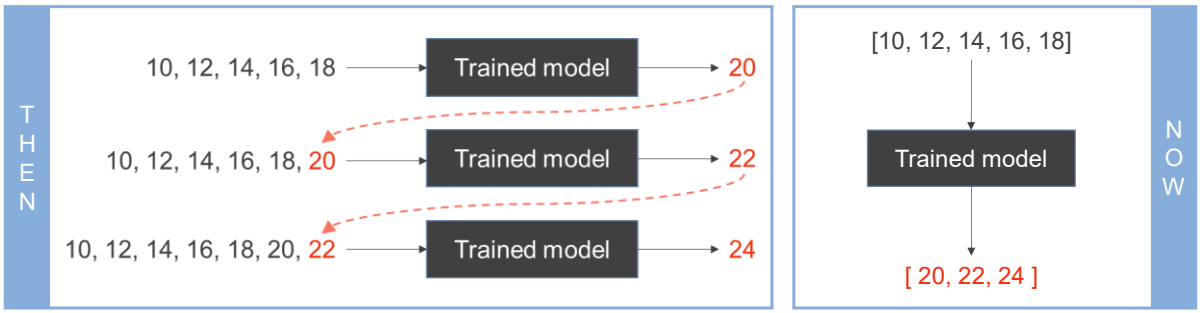
47

EX3: Guess the next three numbers

- Again, we use a simple sequence of $x_n = x_{n-1} + 2$.
- Summary:
  - Reuse the model of "predicting the next number" from the previous example. Append one predicted number to the input sequence, and feed the appended sequence back to the model to make another prediction repeatedly
  - Be careful, denormalize and normalize y properly before appending it to the end of the x input sequence

48

EX4: Guess the next three numbers (revisited 1)

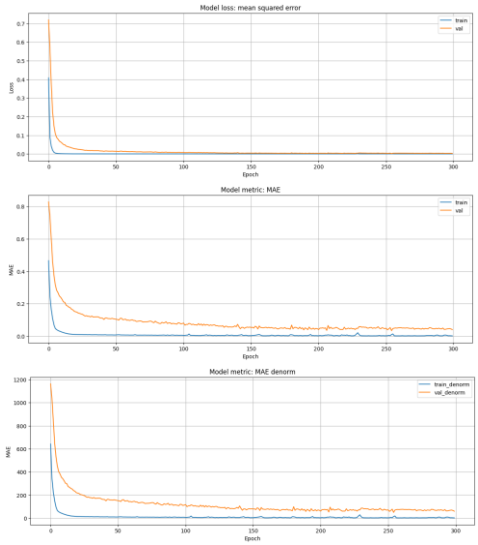


- Summary:
  - Change the model from predicting one scalar value at a time (a.k.a., sequence-to-scalar) to predicting one vector (containing many scalars) at a time (a.k.a., sequence-to-vector)

49

EX4: Guess the next three numbers (revisited 1)

- **Result discussion:**
  - Train MAE small (≈ 1.xx after denormalization) ✅
  - Validation MAE large (≈ 60–70 after denormalization) ❌, but flat, not increasing.
    - The model can learn the training region well, but it cannot generalize to the validation region.
    - It looks like overfit (train ≪ val error). But the classic overfit is when the validation error grows as training continues.
    - In this case, training and validation come from different ranges. Even with a perfect fit on training, validation will stay bad. It's not overfitting, it's **train–val distribution mismatch** due to trend.
  - Suggested fix:
    - Predict changes (deltas or returns) instead of absolute values
    - If predicting absolute values is required, use detrending or rolling normalization



50

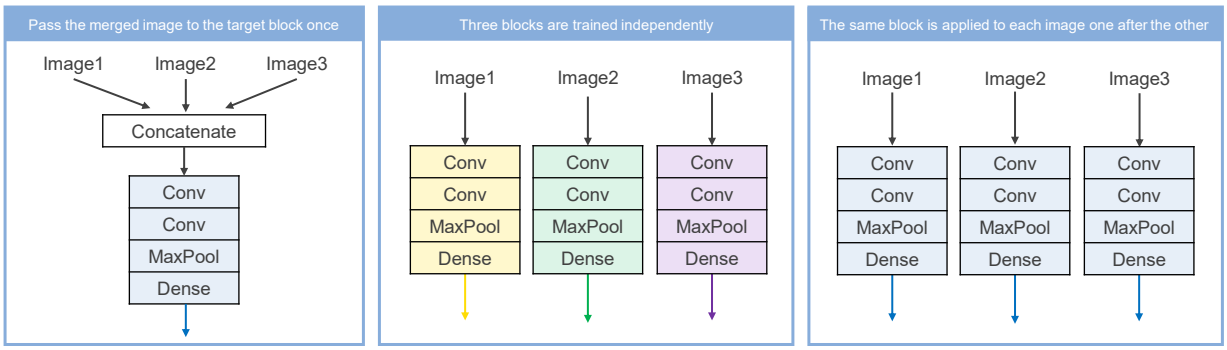
EX5: Guess the next three numbers (revisited 2)

- Change to a sequence-to-sequence model that predicts the next three numbers at each and every time step.
  - At time_step=0, the model outputs a vector containing the predictions for time steps 1 to 3.
  - At time_step=1, the model outputs a vector containing the predictions for time steps 2 to 4.
  - At time_step=2, the model outputs a vector containing the predictions for time steps 3 to 5.
  - …
- In this way, the loss will contain a term for the output of the RNN at each and every time step, not just the output at the last time step.
  - Meaning that there will be many more error gradients flowing through the model (not only flow through time but also flow from the output of each time step)
  - Hence, help both stabilize and speed up training
- To make a sequence-to-sequence model:
  1. Set `return_sequences=True` in all recurrent layers (including the last one)
  2. Apply the output `Dense(3)` layer at every time step, using `keras.layers.TimeDistributed`

51

keras.layers.TimeDistributed

- `TimeDistributed` is a wrapper layer that can be applied to any single layer or any group of layers.
- `TimeDistributed(♣)` means applying the same ♣ separately to each input one after the other.
- For example, to analyze three continuous frames from a video:

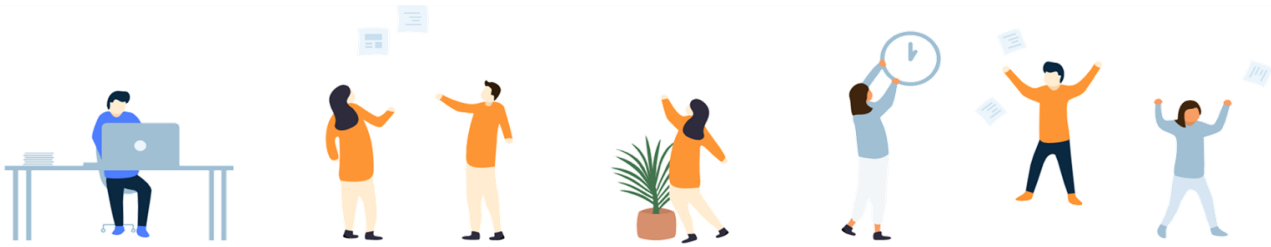


52

keras.layers.TimeDistributed

- The `keras.layers.Dense` actually supports sequences as inputs. Hence, calling `TimeDistributed(Dense(3))` is the same as calling `Dense(3)`.
  - But for clarity, the previous example remains the term `TimeDistributed(Dense(3))` to make it clear that the `Dense` layer is applied independently at each time step, and that the model will output a sequence, not just a single vector.

- Further reading:
  - (23JUL2019) https://medium.com/smileinnovation/how-to-work-with-time-distributed-data-in-a-neural-network-b8b39aa4ce00



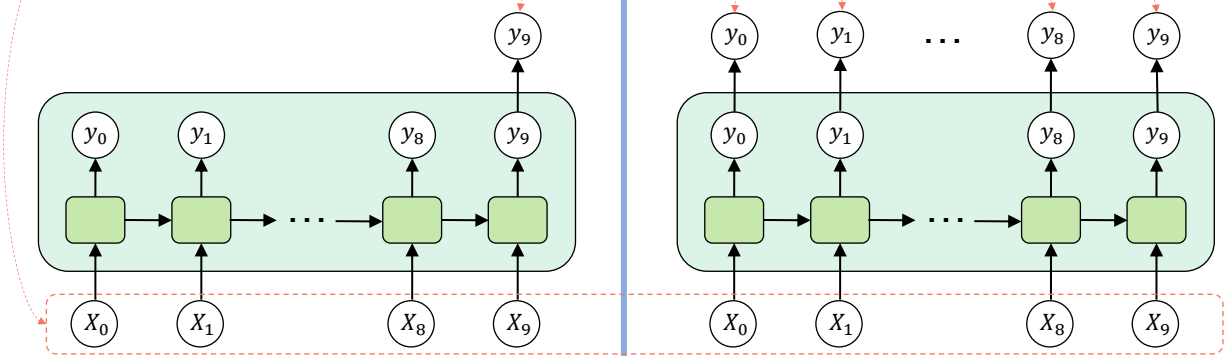
53

```
model.add( SimpleRNN( units=20 ) )
```

```
model.add( SimpleRNN( units=20,
                      return_sequences=True ) )
```

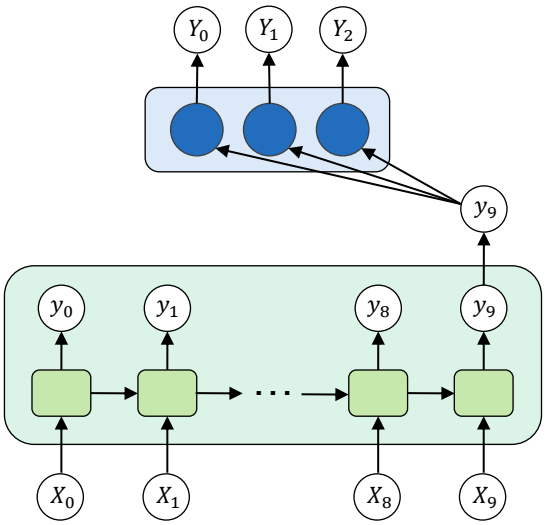
y_i is a 1D vector of shape (20,)

$X = [X_0, X_1, \dots, X_9]$ is an input sequence of shape (time_steps=10,input_dim=1)

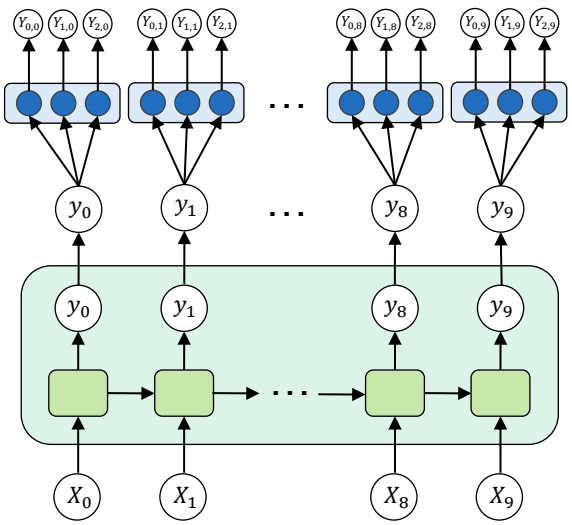


54

```
model.add( SimpleRNN( units=20 ) )
model.add( Dense(3) )
```



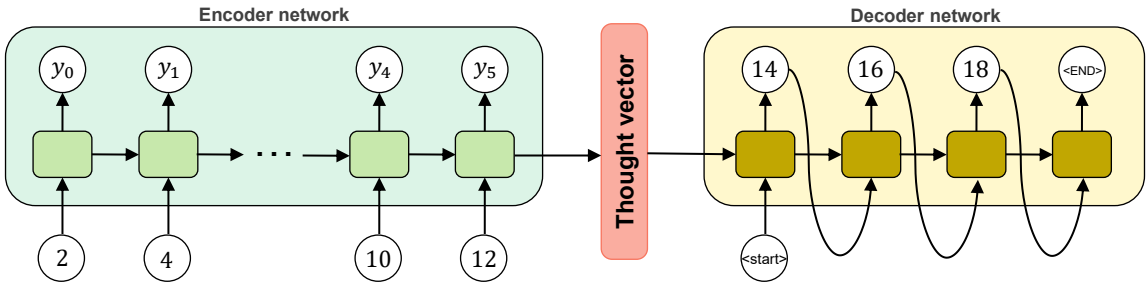
```
model.add( SimpleRNN( units=20,
                      return_sequences=True ) )
model.add( TimeDistributed( Dense(3) ) )
```



55

Encoder-Decoder architecture

- In the two previous examples of “Guess the next three numbers (revisited 1 and 2)”, we use a single RNN network to learn two things at the same time—understanding an input sequence **and** generating an output sequence. This is too much for a single network, hence, generated output sequences may not be precise.
- Popular (but also much more complicated) solution for seq2seq prediction is to use an **Encoder-Decoder architecture** where two networks are sequentially connected; the first network encodes an input sequence whereas the second network generates an output sequence.

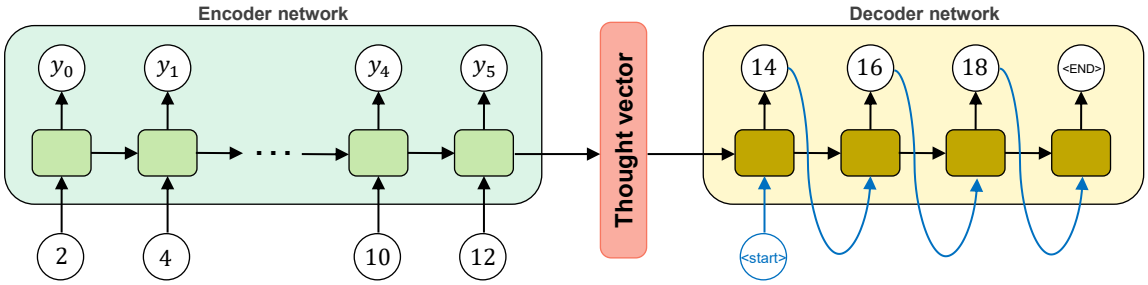


56

Encoder-Decoder architecture

Alternative 1: Closed-loop Training

- Except for the first time step that uses a special start token as input, in other time steps of the decoder, use decoder's output at time $t-1$ as an input of decoder at time t.

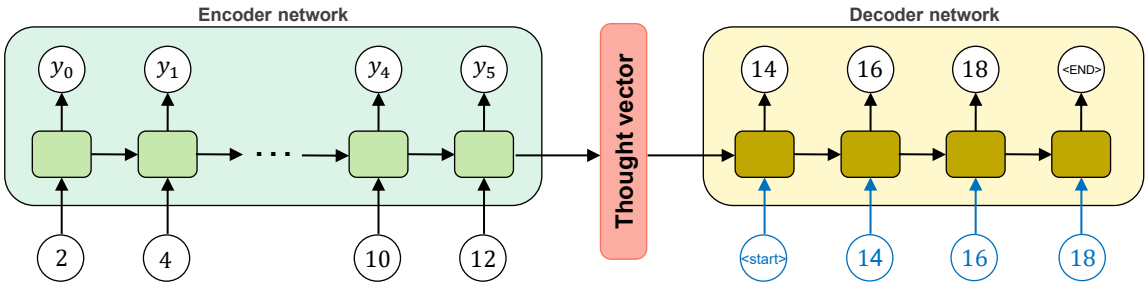


57

Encoder-Decoder architecture

Alternative 2: Teacher Forcing (1989)

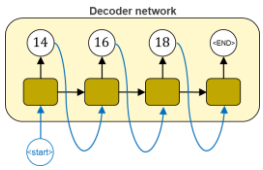
- Teacher Forcing is fast and effective for training RNNs.
- Teacher Forcing is a common training strategy for language models.



58

Encoder-Decoder architecture

Alternative 2: Teacher Forcing (1989)



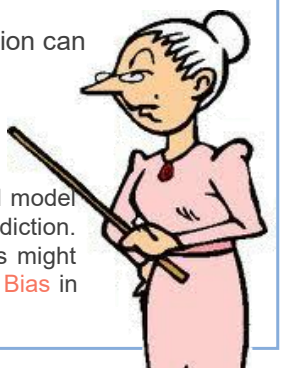
- Models that have recurrent connections from their outputs leading back into the model may be trained with Teacher Forcing.
- Originally, Teacher Forcing was described and developed as an alternative technique to BPTT for training RNNs.
  - If output-to-input feedback loops are the only recurrent loops, the different time steps of the unrolled computational graph can be decoupled completely.
  - With the complete teacher forcing, this means that gradients do not propagate over time (i.e., no BPTT), as the effect of the weights of the last time step on the current hidden state is removed.
- **BUT** for generative tasks, Teacher Forcing is often used in conjunction with BPTT.

59

Encoder-Decoder architecture

Alternative 2: Teacher Forcing (1989)

- Why Teacher Forcing?
  - The model is more stable during training and can converge quickly.
    - At the early stages of training, the predictions of the model are very bad. Without Teacher Forcing, the hidden states of the model will be updated by a sequence of wrong predictions, errors will accumulate, and it is difficult for the model to learn from that.
  - Teacher-forced training works well with ML accelerators as the computation can be parallelized across time.
- Why not Teacher Forcing?
  - Train-test time discrepancy:
    - During inference, since there is usually no ground truth available, the RNN model will need to feed its own previous prediction back to itself for the next prediction. Therefore, there is a discrepancy between training and inference, and this might lead to poor model performance and instability. This is known as Exposure Bias in literature.

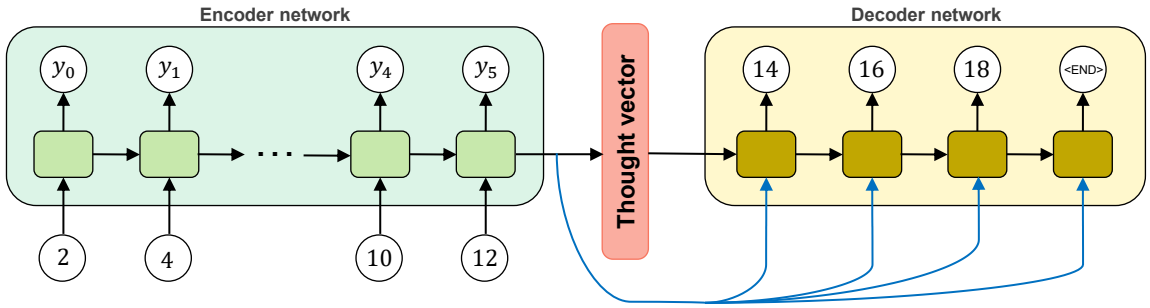


60

Encoder-Decoder architecture

Other

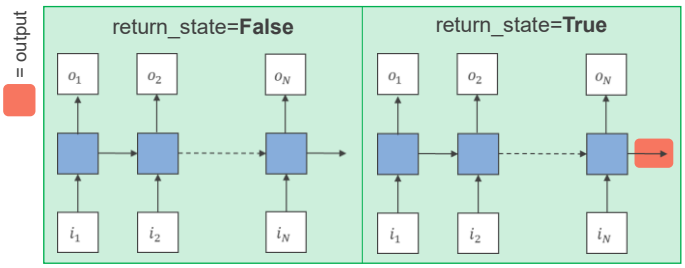
- Copy the last hidden state of encoder and use it as inputs to all time steps of decoder



61

By default, SimpleRNN in keras does not return the hidden state (from the last time step).

```
keras.layers.SimpleRNN ( units,
                         activation='tanh',
                         return_sequences=False ,
                         return_state=False
                       )
```



62

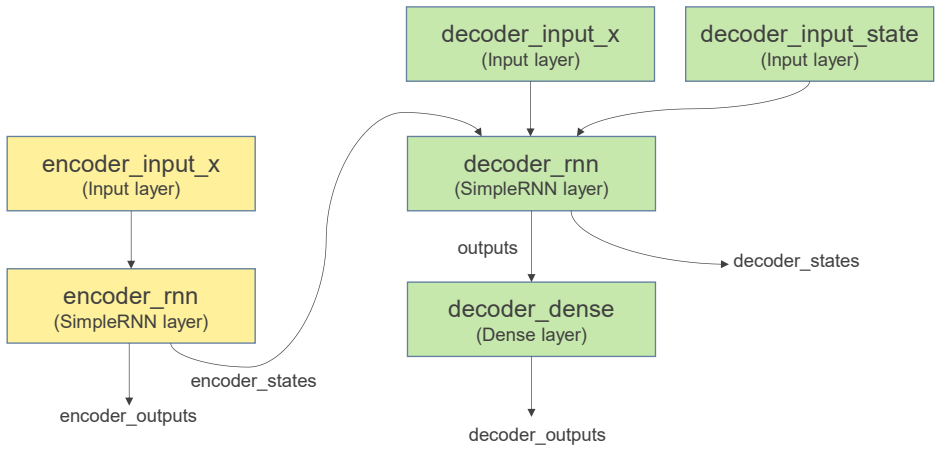
EX6: Guess the next three numbers (revisited 3)

- Use `keras.Model()` to create a complicated network of **Encoder-Decoder architecture**
  - The main network consists of two RNN networks (one as an encoder and the other as a decoder), allowing any number of output's time steps despite the input's time steps.
  - While sharing the same computational graph (and weights), there are three separated variables created with `keras.Model()`—one variable (named `seq2seq_model`) refers to the main **Encoder-Decoder** network to be used for training, the other two variables (named `encoder_model` and `decoder_model`) are the encoder-only and decoder-only networks used for inferencing.
  - Be careful when creating the decoder model as it requires different inputs during training and inferencing.
- Use `keras.utils.plot_model()` to visualize the non-sequential model architecture
- Create a custom callback to save the best `encoder_model` and `decoder_model`
- During inferencing, the decoder-only network is repeatedly called (one call per one time-step prediction).
- Create a loop for autoregressive evaluation

63

EX6: Guess the next three numbers (revisited 3)

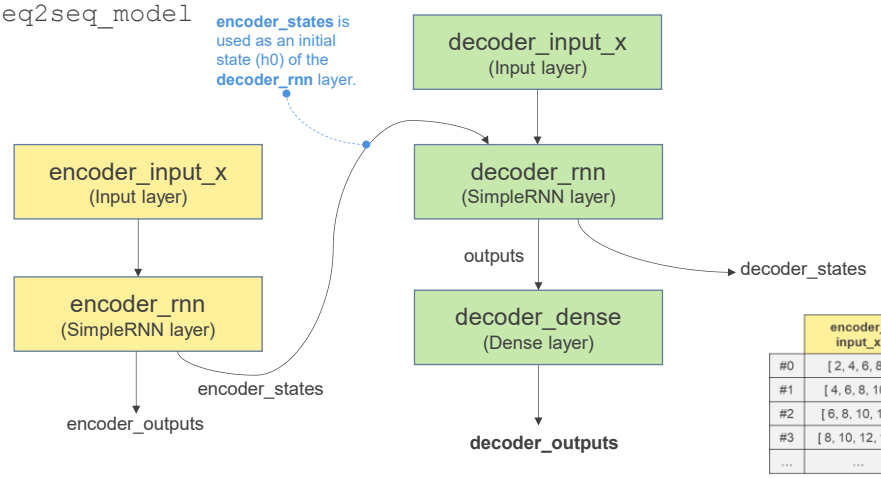
- Overall computational graph:



64

EX6: Guess the next three numbers (revisited 3)

- Training graph:
  - `seq2seq_model`



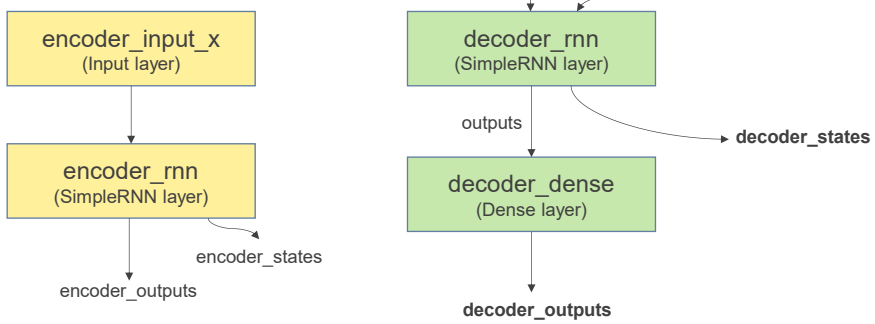
	encoder_ input_x	decoder_ input_x	decoder_ outputs
#0	[2, 4, 6, 8]	[<start>, 10, 12]	[10, 12, 14]
#1	[4, 6, 8, 10]	[<start>, 12, 14]	[12, 14, 16]
#2	[6, 8, 10, 12]	[<start>, 14, 16]	[14, 16, 18]
#3	[8, 10, 12, 14]	[<start>, 16, 18]	[16, 18, 20]
...	...	...	...

65

EX6: Guess the next three numbers (revisited 3)

- Inferencing graph:
  - `encoder_model`
  - `decoder_model`

Prediction	INPUTs		OUTPUTs	
	decoder_ input_x	decoder_ input_state	decoder_ outputs	decoder_ states
#0	<start token>	encoder_states	y_pred0	d0
#1	y_pred0	d0	y_pred1	d1
#2	y_pred1	d1	y_pred2	d2
#3	y_pred2	d2	y_pred3	d3
...	...	...	...	...

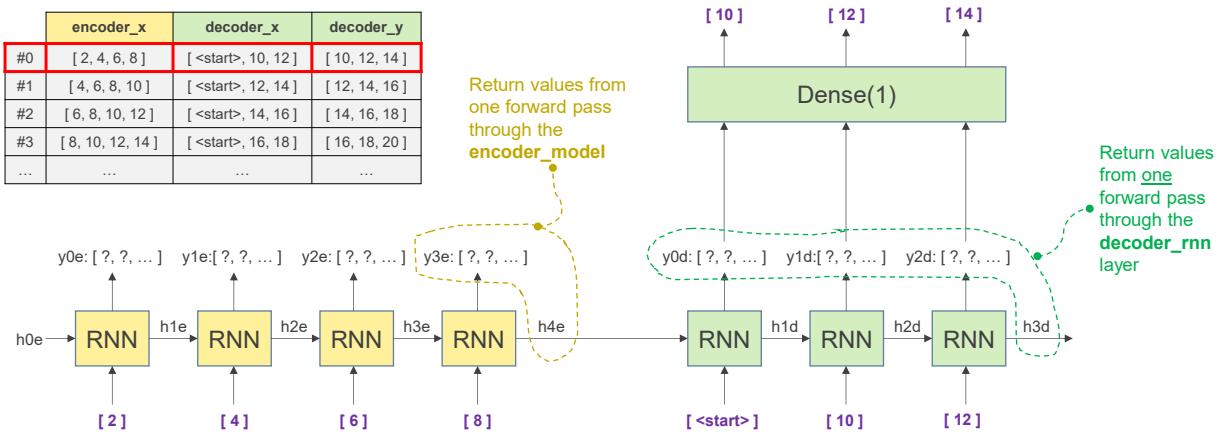


66

EX6: Guess the next three numbers (revisited 3)

- Training: `seq2seq_model`

	encoder_x	decoder_x	decoder_y
#0	[2, 4, 6, 8]	[<start>, 10, 12]	[10, 12, 14]
#1	[4, 6, 8, 10]	[<start>, 12, 14]	[12, 14, 16]
#2	[6, 8, 10, 12]	[<start>, 14, 16]	[14, 16, 18]
#3	[8, 10, 12, 14]	[<start>, 16, 18]	[16, 18, 20]
…	…	…	…



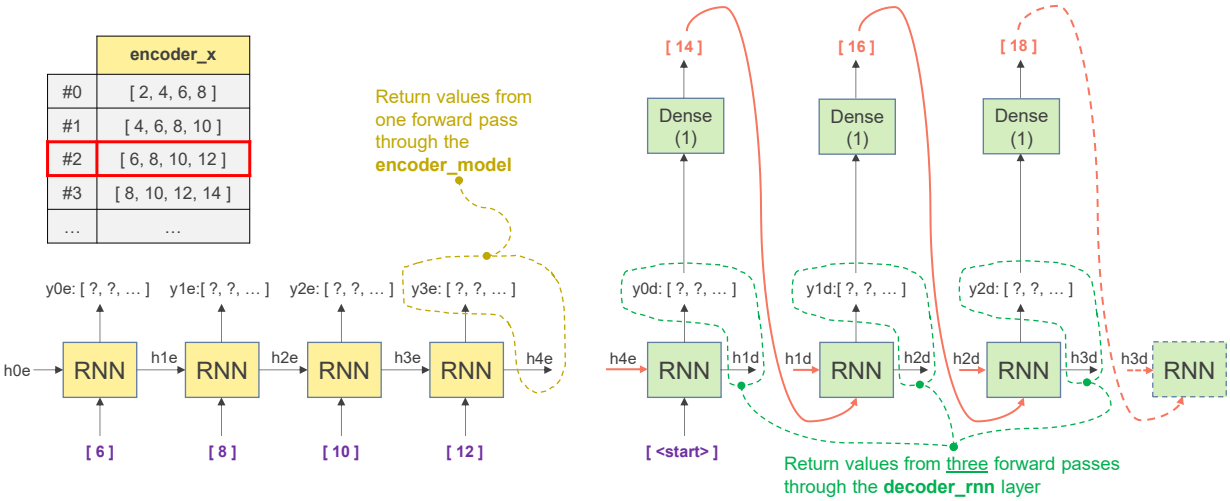
67

EX6: Guess the next three nur

Auto-regressive model refers to a time series model that uses observations from previous time steps as input to a regression equation to predict the value at the next time step.

- Inferencing: `encoder_model`, `decoder_model`

	encoder_x
#0	[2, 4, 6, 8]
#1	[4, 6, 8, 10]
#2	[6, 8, 10, 12]
#3	[8, 10, 12, 14]
…	…



68

69